c) Vary the control variable over the following sequence of values: 2, 5, 8, 11, 15, 17.

```
for (int j = 2; j <= 17; j += 3)
```

d) Vary the control variable from 20 to 2 in increments of -2.

```
for (int i = 20; i >= 2; i -= 2)
```

Answer: c) is incorrect. The `for` statement actually generates the sequence 2, 5, 8, 11, 14, 17. It does not generate the value 15 in the original series.

4.6 switch Multiple-Selection Statement

In Chapter 3, we discussed the `if` single-selection and the `if...else` double-selection statements. Occasionally, an algorithm will contain a series of decisions that test a variable or expression separately for each of the integer values it may assume, then perform different actions. This is called multiple selection. C provides the `switch` multiple-selection statement to handle such decision making.

The `switch` statement consists of a series of `case` labels, an optional `default` case and statements to execute for each case. Figure 4.5 uses `switch` to count the number of each different letter grade students earned on an exam.

```c
// fig04_05.c
// Counting letter grades with switch.
#include <stdio.h>

int main(void) {
   int aCount = 0;
   int bCount = 0;
   int cCount = 0;
   int dCount = 0;
   int fCount = 0;

   puts("Enter the letter grades.");
   puts("Enter the EOF character to end input.");
   int grade = 0; // one grade

   // loop until user types end-of-file key sequence
   while ((grade = getchar()) != EOF) {

      // determine which grade was input
      switch (grade) { // switch nested in while
         case 'A': // grade was uppercase A
         case 'a': // or lowercase a
            ++aCount;
            break; // necessary to exit switch
         case 'B': // grade was uppercase B
         case 'b': // or lowercase b
            ++bCount;
            break;
```

Fig. 4.5 | Counting letter grades with `switch`. (Part 1 of 2.)

```
            case 'C': // grade was uppercase C
            case 'c': // or lowercase c
               ++cCount;
               break;
            case 'D': // grade was uppercase D
            case 'd': // or lowercase d
               ++dCount;
               break;
            case 'F': // grade was uppercase F
            case 'f': // or lowercase f
               ++fCount;
               break;
            case '\n': // ignore newlines,
            case '\t': // tabs,
            case ' ': // and spaces in input
               break;
            default: // catch all other characters
               printf("%s", "Incorrect letter grade entered.");
               puts(" Enter a new grade.");
               break; // optional; will exit switch anyway
         } // end switch
      } // end while

      // output summary of results
      puts("\nTotals for each letter grade are:");
      printf("A: %d\n", aCount);
      printf("B: %d\n", bCount);
      printf("C: %d\n", cCount);
      printf("D: %d\n", dCount);
      printf("F: %d\n", fCount);
   }
```

```
Enter the letter grades.
Enter the EOF character to end input.
a
b
c
C
A
d
f
C
E
Incorrect letter grade entered. Enter a new grade.
D
A
b
^Z ——— Not all systems display a representation of the EOF character

Totals for each letter grade are:
A: 3
B: 2
C: 3
D: 2
F: 1
```

Fig. 4.5 | Counting letter grades with `switch`. (Part 2 of 2.)

Reading Character Input

In the program, the user enters students' letter grades. In the `while` header (line 17),

```
while ((grade = getchar()) != EOF)
```

the parenthesized assignment `(grade = getchar())` executes first. The `getchar` function (from `<stdio.h>`) reads one character from the keyboard and stores that character in the integer variable `grade`. Characters are normally stored in **`char`** variables. However, C can store characters in variables of any integer type because characters are usually represented as one-byte integers in the computer. Function `getchar` returns as an `int` the character that the user entered. We can treat a character as either an integer or a character, depending on its use. For example, the statement

```
printf("The character (%c) has the value %d.\n", 'a', 'a');
```

uses the conversion specifications `%c` and `%d` to print the character `'a'` and its integer value. The result is

```
The character (a) has the value 97.
```

Characters can be read with `scanf` by using the conversion specification `%c`. The integer 97 is the numerical representation of the character `'a'` in the computer. Many computers today use the Unicode® character set. Appendix B contains the **ASCII (American Standard Code for Information Interchange) character set** and its numeric values. ASCII is a subset of Unicode.

Assignments Have Values

Assignments as a whole actually have a value. The value of the assignment expression `grade = getchar()` is the character that's returned by `getchar` and assigned to the variable `grade`. The fact that assignments have values can be useful for setting several variables to the same value. For example,

```
a = b = c = 0;
```

first evaluates the assignment `c = 0` (because the `=` operator groups from right-to-left). The variable `b` is then assigned the value of the assignment `c = 0` (which is 0). Then, the variable `a` is assigned the value of the assignment `b = (c = 0)` (which is also 0).

The value of the assignment `grade = getchar()` is compared with the value of `EOF` (a symbol whose acronym stands for "end of file"). We use `EOF` (which normally has the value `-1`) as the sentinel value. The user types a system-dependent keystroke combination to mean "end of file"—i.e., "I have no more data to enter." `EOF` is a symbolic integer constant defined in the `<stdio.h>` header (we'll see in Chapter 6 how symbolic constants are defined). If the value assigned to `grade` is equal to `EOF`, the program terminates.

We represent characters in this program as `int`s because `EOF` has an integer value (again, normally `-1`). Testing for the symbolic constant `EOF`, rather than `-1`, makes programs more portable. The C standard states that `EOF` is a negative integral value (but not necessarily `-1`). Thus, `EOF` could have different values on different systems.

Entering the EOF Indicator

The keystroke combinations for entering EOF (end of file) are system dependent. On Linux/UNIX/macOS systems, the `EOF` indicator is entered by typing on a line by itself

Ctrl + *d*

This notation means to simultaneously press both the *Ctrl* key and the *d* key. On other systems, such as Microsoft Windows, the `EOF` indicator can be entered by typing

Ctrl + *z*

You also need to press *Enter* on Windows.

The user enters grades at the keyboard. When the *Enter* key is pressed, the characters are read by function `getchar` one at a time. If the character entered is not equal to `EOF`, the `switch` statement (lines 20–49) executes.

switch Statement Details

Keyword `switch` is followed by the variable name `grade` in parentheses. This is called the **controlling expression**. The `switch` compares this expression's value with each of the **case labels**. Each `case` can have one or more actions, but braces are not required around multiple actions in a given `case`.

Assume the user has entered the letter `C` as a grade. When the `switch` compares `C` to each `case`, if a match occurs (`case 'C':`), the statements for that `case` execute. For the letter `C`, the `switch` increments `cCount` by `1` (line 31), then the `break` statement (line 32) exits the `switch` immediately, causing program control to continue with the first statement after the `switch` statement.

We use a `break` statement here because the `case`s in a `switch` statement would otherwise run together. Without `break` statements, each time a match occurs, all the remaining `case`s' statements will execute. (This feature—called fallthrough—is rarely useful, although it's perfect for compactly programming Exercise 4.38—the iterative song "The Twelve Days of Christmas"!) Forgetting a `break` statement when one is needed in a `switch` statement is a logic error. ERR

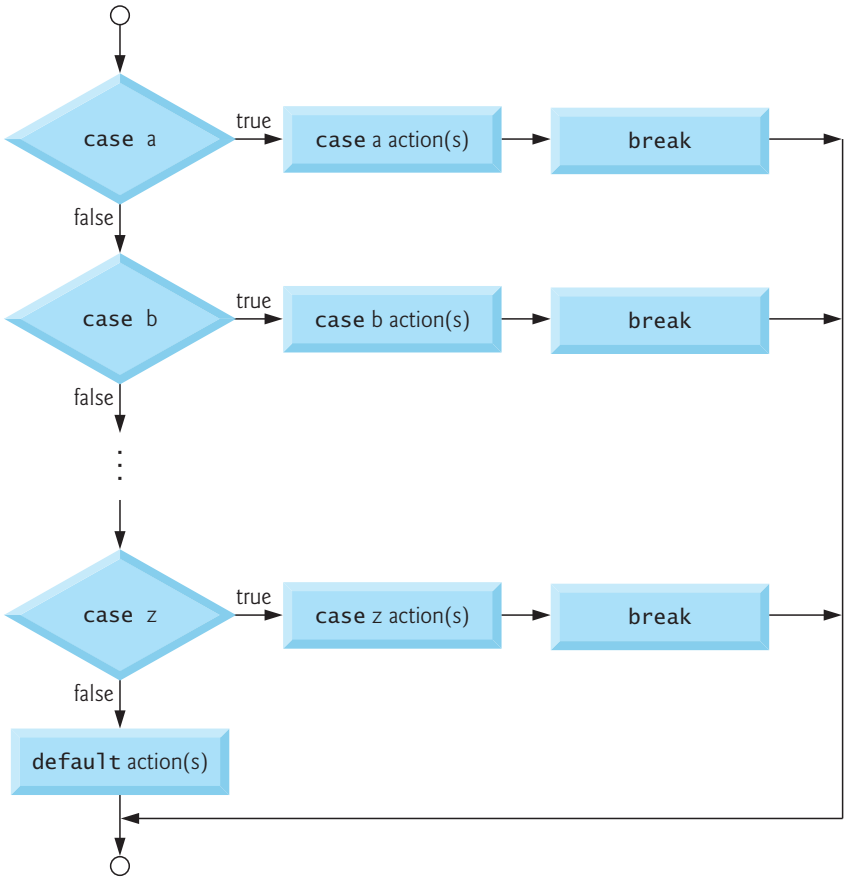
default Case

If no match occurs, the `default` case executes. In this program, it displays an error message. You should always include a `default` case; otherwise, values not explicitly tested in a `switch` will be ignored. The `default` case helps prevent this by focusing you on the need to process exceptional conditions. Sometimes no `default` processing is needed.

Although the `case` clauses and the `default` case clause in a `switch` statement can occur in any order, it's common to place the `default` clause last. When the `default` clause is last, the `break` statement isn't required. But many programmers include this `break` for clarity and symmetry with other cases.

switch Statement Flowchart

The following `switch` multiple-selection-statement flowchart makes it clear that each `case`'s `break` statement immediately exits the `switch` statement.



Ignoring Newline, Tab and Blank Characters in Input

In the `switch` statement of Fig. 4.5, the lines

```
case '\n': // ignore newlines,
case '\t': // tabs,
case ' ': // and spaces in input
   break;
```

cause the program to skip newline, tab and blank characters. Reading characters one at a time can cause problems. To have the program read the characters, you must send them to the computer by pressing *Enter*. This places the newline character in the input after the character we wish to process.

Often, this newline (and other whitespace characters) must be specifically ignored to make the program work correctly. The preceding cases in our `switch` statement prevent the error message in the `default` case from being printed each time a newline, tab or space is encountered in the input. Each input in this example causes two iterations of the loop—the first for a letter grade and the second for `'\n'`. Listing several `case` labels with no intervening statements means that the same actions occur for each of the `case`s.

Constant Integral Expressions

When using the `switch` statement, remember that each `case` can test only a **constant integral expression**. The expression can be any combination of character constants and integer constants that evaluates to a constant integer value. A character constant can be represented as the specific character in single quotes, such as `'A'`. Characters must be enclosed within single quotes to be recognized as character constants—characters in double quotes are recognized as strings. Integer constants are simply integer values. In our example, we used character constants.

Notes on Integral Types

Portable languages like C must have flexible data-type sizes. Applications may need integers of various sizes. C provides several data types to represent integers. In addition to `int` and `char`, C provides types `short int` (which can be abbreviated as `short`) and `long int` (which can be abbreviated as `long`). There also are `unsigned` variations of all the integral types that represent non-negative integer values. In Section 5.14, we'll see that C also provides type `long long int` (which can be abbreviated as `long long`).

The C standard specifies the minimum range of values for each integer type. The actual range may be greater, depending on the implementation. For `short ints`, the minimum range is –32767 to +32767. For most integer calculations, `long ints` are sufficient. The minimum range of values for `long ints` is –2147483647 to +2147483647. An `int`'s range is greater than or equal to that of a `short int` and less than or equal to that of a `long int`. On many of today's platforms, `ints` and `long ints` represent the same range of values. The data type `signed char` can represent integers in the range –127 to +127 or any of the ASCII character set. See Section 5.2.4.2 of the C standard document for the complete list of `signed` and `unsigned` integer-type minimum ranges.

Self Check

1 *(Fill-In)* Occasionally, an algorithm will contain a series of decisions in which a variable or expression is tested separately for each of the constant integral values it may assume, and different actions are taken. This is called _____.

Answer: multiple selection.

2 *(Multiple Choice)* Which of the following statements a), b) or c) is *false*?

a) The value of an assignment is the value assigned to the variable on the left of the `=`.

b) The value of the assignment expression `grade = getchar()` is the character that's returned by `getchar` and assigned to the variable `grade`.

c) The following statement sets variables `a`, `b` and `c` to `0`:

```
0 = a = b = c;
```

d) All of the above statements are *true*.

Answer: c) is *false*. The correct statement is:

```
a = b = c = 0;
```